CO326 — Computer Systems Engineering & Industrial Networks

Motor Thermal Monitoring Digital Twin

with Edge Anomaly Detection and Predictive Maintenance

Name	Index Number
Haritha Bandara	E/20/037
Yohan Senanayake	E/20/363
Chamodi Senarathne	E/20/365
Janith Wanasinghe	E/20/420

Group 10
Department of Computer Engineering
University of Peradeniya

May 2026

Abstract

This report describes the design and implementation of a real-time motor thermal monitoring system built on an Industrial IoT architecture. The system uses an ESP32 microcontroller with a Light Dependent Resistor (LDR) as a proxy temperature sensor, a Docker-based edge AI stack for anomaly classification, and a Node-RED dashboard for operator control. Two complementary AI subsystems run in parallel from the same sensor stream: a reactive edge AI module that classifies each reading as NORMAL, WARNING, or CRITICAL and immediately drives a cooling fan relay, and a predictive maintenance service that analyses a sliding window of historical readings to estimate how many minutes remain before the motor crosses the warning threshold. Four interchangeable classification algorithms are provided for the reactive module — Z-score baseline, Decision Tree, Isolation Forest, and Random Forest — all behind a single-line swap interface. The predictive service offers a linear slope extrapolation model for fast verification and an LSTM neural network as the high-accuracy upgrade. Alert data is forwarded to a cloud SCADA broker and stored in InfluxDB, with a Grafana digital twin dashboard providing live visualisation of both real-time status and predicted risk.

Contents

1 Introduction **4**

2 System Architecture **4**
2.1 Overview . . . 5
2.2 MQTT Topic Design . . . 5
2.3 Docker Compose Stack . . . 6
2.4 Cloud SCADA Integration . . . 7

3 Hardware Implementation **7**
3.1 Component Selection and Circuit Design . . . 7
3.2 LDR Orientation and Temperature Mapping . . . 9
3.3 ESP32 Firmware . . . 9
3.4 Software Simulator . . . 10

4 Software Stack and Infrastructure **11**
4.1 MQTT Broker — Mosquitto . . . 11
4.2 Node-RED HMI Dashboard . . . 11
4.3 Initial Docker Compose Structure . . . 12

5 Edge AI Module — Core Logic **12**
5.1 Design Philosophy . . . 12
5.2 Rolling Window . . . 13
5.3 Fan Control Hysteresis . . . 13
5.4 AUTO and MANUAL Mode . . . 14
5.5 Alert Payload . . . 14

6 Reactive Classification Algorithms **15**
6.1 Why Four Algorithms? . . . 15
6.2 Shared Feature Engineering . . . 15
6.3 Algorithm 0 — Z-Score Baseline . . . 16
6.4 Algorithm 1 — Decision Tree . . . 16
6.5 Algorithm 2 — Isolation Forest . . . 17
6.6 Algorithm 3 — Random Forest . . . 18
6.7 Summary Comparison . . . 19

7 Predictive Maintenance Service **19**
7.1 Motivation and Scope . . . 19
7.2 New MQTT Topic and Output . . . 20
7.3 Sliding Buffer . . . 20
7.4 Phase 1 — Baseline Model (Linear Slope Extrapolation) . . . 20
7.5 Phase 2 — LSTM Model . . . 21
7.5.1 Architecture . . . 21
7.5.2 Training Locally, Not in Docker . . . 22
7.5.3 Synthetic Training Data . . . 22
7.6 Swappable Model Registry . . . 23
7.7 InfluxDB, Grafana, and Node-RED Integration . . . 23

7.8 Key Result — Early Warning Lead Time 24

8 Challenges and Solutions **25**
8.1 ESP32 ADC2 Conflict with WiFi 25
8.2 Z-Score Instability on Cold Start 26
8.3 Isolation Forest Silent Warm-Up 26
8.4 Training Data Generation and Class Imbalance 26
8.5 Asynchronous Data vs. Sequential Input 26
8.6 InfluxDB Init Token Bootstrap 26
8.7 Node-RED Flow Merge Conflicts 27
8.8 tflite-runtime Architecture Compatibility 27

9 Member Contributions **27**

10 Conclusion and Future Work **28**
10.1 What We Built 29
10.2 Future Work 29

1 Introduction

Written by: Yohan Senanayake

Industrial motors are among the most critical and failure-prone components in manufacturing environments. Thermal stress is the leading cause of insulation breakdown, bearing seizure, and catastrophic motor failure. In many facilities, monitoring is still done manually or through simple alarm thresholds wired to a single thermocouple — a setup that reacts to failures rather than anticipating them. The cost of an unplanned motor shutdown in a production line can reach tens of thousands of dollars within hours, making early detection an obvious priority.

This project addresses that gap by building a Motor Thermal Digital Twin: a software replica of a motor's thermal state that runs in real time alongside the physical hardware. Instead of just triggering an alarm when the temperature crosses a threshold, our system maintains a continuous model of the motor's thermal behaviour, classifies every new reading with a machine learning algorithm, controls a cooling fan automatically, and — critically — estimates whether the motor is trending toward overheating before it actually gets there.

We decided to use an ESP32 microcontroller with an LDR sensor rather than a conventional thermistor, partly to keep the circuit simple and partly because the LDR provides a hands-on way to simulate thermal events in the lab: covering the sensor raises the ADC reading, which the firmware maps to a rising temperature. This is admittedly a proxy and not a true thermal measurement, but it gives us a real, controllable hardware signal to work with — which matters for demonstrating the full system end-to-end.

The software stack is built entirely on open-source, containerised tools: Eclipse Mosquitto for MQTT messaging, Python for both the edge AI and predictive service, Node-RED for the operator dashboard, InfluxDB for time-series storage, and Grafana for the digital twin visualisation. Everything runs with a single `docker compose up` command.

2 System Architecture

Written by: Haritha Bandara

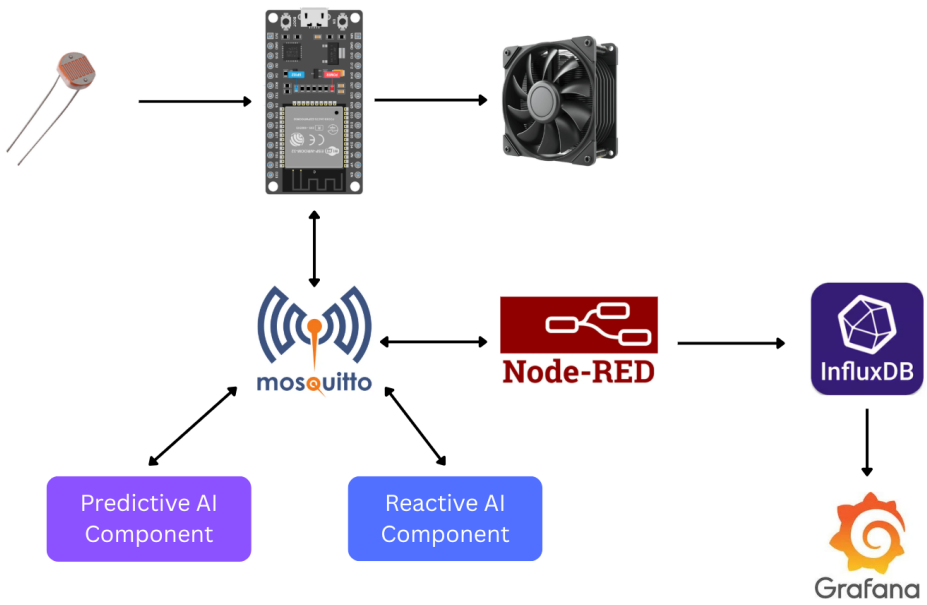

Figure 1: System architecture. The reactive AI path (right) drives immediate fan control; the predictive AI path (left) publishes forward-looking risk scores. Both paths originate from the same sensor stream.

2.1 Overview

The system follows a layered edge-to-cloud architecture with five clearly separated concerns: sensing, reactive intelligence, predictive intelligence, operator interaction, and long-term storage. The design decision that stands out is the the clean separation between the reactive and predictive AI layers. Both containers subscribe to the same sensor topic, but they operate independently and never interfere with each other. The reactive container (`python-edge`) is the only one that can publish fan commands. The predictive container (`python-predict`) is strictly read-only from a control perspective — it publishes only to its own MQTT topic and never touches the fan relay. This means a crash or miscalibration in the predictive service cannot affect the fan control logic that keeps the motor safe.

2.2 MQTT Topic Design

MQTT is used as the messaging backbone because its lightweight publish-subscribe model is well suited for IoT environments where publishers and subscribers need to be decoupled. 5 Topics were defined with clear ownership. (Table 1).

Table 1: MQTT topic assignments

Topic	**Publisher**	**Subscriber**	**Purpose**
`sensors/.../data`	ESP32	Edge AI, Predictive	Raw temperature every 2 s
`alerts/.../status`	Edge AI	Node-RED, Cloud	Anomaly status + score
`control/.../fan`	Edge AI / HMI	ESP32	Fan relay command
`control/.../mode`	Node-RED	Edge AI	AUTO / MANUAL (retained)
`predict/.../risk`	Predictive	Node-RED, InfluxDB	Risk score 0–1, ETA

(Full topic paths use the prefix `group10/motorTemp`.)

One design that was enforced throughout: raw sensor readings being in local. The ESP32 publishes temperature data only to the local Mosquitto broker. The edge AI then publishes processed alerts — not raw readings — to both the local broker and the cloud SCADA broker. This reduces outbound bandwidth and avoids sending sensitive operational data off-site.

2.3 Docker Compose Stack

The `LDR-Hardware-simulation/docker-compose.yml` started with three core services, and expanded to seven as the predictive maintenance and storage layers were added by Chamodi (see Section 7). Table 2 shows the complete final state.

Table 2: Docker Compose services (final state after all team additions)

Service	Image	Port	Role
`mqtt`	eclipse-mosquitto:2.0	1883	MQTT broker
`node-red`	nodered/node-red:latest	1880	Operator HMI
`python-edge`	custom Python 3.11	—	Reactive edge AI
`python-predict`	custom Python 3.11	—	Predictive maintenance
`influxdb`	influxdb:2.7	8086	Time-series historian
`grafana`	grafana/grafana:latest	3000	Digital twin dashboards
`python-device`	custom Python 3.11	—	Software simulator

The `python-device` service is placed under a Docker Compose `simulation` profile. When the real ESP32 is connected, this service stays off. When testing on a laptop without hardware, running `docker compose -profile simulation up` starts the software simulator alongside everything else.

2.4 Cloud SCADA Integration

The edge AI maintains two separate MQTT client instances: one connected to the local Mosquitto broker (guaranteed delivery) and one connected to the cloud SCADA broker. Every alert is published to the local broker first. The cloud publish is then attempted inside a `try/except` block — if the cloud broker is unreachable, the exception is caught, logged, and discarded. The fan control loop never waits for the cloud publish to complete. This means the system continues to protect the motor safely even with no internet connectivity.

3 Hardware Implementation

Written by: Janith Wanasinghe

3.1 Component Selection and Circuit Design

When hardware choice is concidered, the obvious choice for a temperature sensor would have been a thermistor or a dedicated digital sensor like the DS18B20. Something that

would allow anomalies to be physically triggered in the lab without needing a heat source was wanted, and an LDR provided exactly that. Covering it with a finger or a piece of dark paper raises the ADC reading, which the firmware maps to a higher temperature — it is a proxy, but a controllable one that lets you demonstrate the full response cycle in seconds.

The bill of materials is minimal:

- ESP32 Dev Module (38-pin)
- LDR (photoresistor) with 10 kΩ pull-down resistor, forming a voltage divider
- 5 V relay module (active-HIGH logic)
- Small DC fan (or an LED as a visual substitute for the relay output)

Table 3: GPIO pin assignments

Component	**GPIO**	**Direction**	**Notes**
LDR (voltage divider midpoint)	34	Analog input	ADC1 bank — no WiFi conflict
Relay module	12	Digital output	HIGH = ON, LOW = OFF (active-HIGH)

GPIO 34 was deliberately chosen for the LDR. The ESP32's ADC2 pins are shared with the WiFi radio, so any GPIO in the ADC2 group returns unreliable readings when WiFi is active. GPIO 34 belongs to ADC1, which is independent of the WiFi peripheral. That one detail tripped us up early on — Initially GPIO 13 was wired with LDR and an afternoon was spent debugging ADC readings that would jump unpredictably every time the MQTT connection was established. Moving to GPIO 34 fixed it immediately.

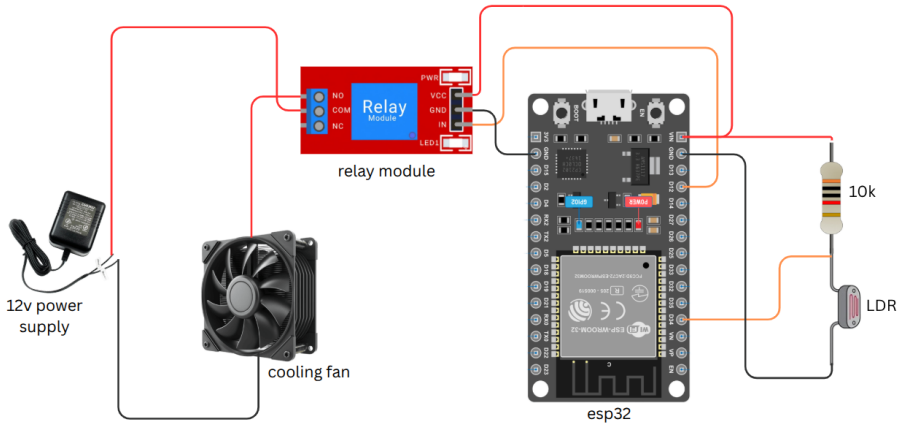

Figure 2: LDR voltage divider and relay circuit on the ESP32 Dev Module.

3.2 LDR Orientation and Temperature Mapping

The LDR sits at the top of the voltage divider with a 10 kΩ resistor to ground. In ambient light the LDR resistance is low, pulling the midpoint voltage down and producing a low ADC reading. When the LDR is covered, its resistance rises sharply, raising the midpoint voltage and pushing the ADC reading toward 4095.

The firmware maps this 0–4095 ADC range to a temperature range of 40–100 °C using a simple linear formula:

$$T = 40.0 + \frac{\text{ADC}}{4095} \times 60.0 \tag{1}$$

An uncovered LDR in normal office lighting typically reads around ADC 1800–2200, mapping to roughly 65–72 °C — a plausible motor surface temperature under light load. Fully covering the sensor produces ADC values above 3800, mapping to temperatures above 95 °C and reliably triggering the CRITICAL alarm.

3.3 ESP32 Firmware

The firmware is written in C++ using the Arduino framework, with three library dependencies: `WiFi.h` (built-in), `PubSubClient` by Nick O'Leary for MQTT, and `ArduinoJson` by Benoît Blanchon for JSON serialisation.

The `loop()` function does three things in sequence on every iteration:

1. Checks WiFi and MQTT connectivity and reconnects if either has dropped.
2. Calls `mqttClient.loop()`, which processes any incoming fan commands in a few milliseconds.

3. Every 2 000 ms, reads the LDR ADC, maps it to a temperature float using Equation 1, serialises a JSON payload, and publishes it.

The fan command handler is kept intentionally simple:

Listing 1: Fan command callback in firmware.ino

```
void mqttCallback(char* topic, byte* payload, unsigned int
   length) {
  StaticJsonDocument<64> doc;
  if (deserializeJson(doc, payload, length)) return;
  const char* cmd = doc["command"];
  fanState = (strcmp(cmd, "ON") == 0);
  digitalWrite(RELAY_PIN, fanState ? HIGH : LOW);
}
```

The firmware was deliberately kept lean, with local alarm logic, displays, and persistent storage all being omitted. All intelligence is hosted within Python containers on the host machine. Consequently, the firmware almost never requires modification; the addition of a new algorithm or the adjustment of a threshold only necessitates the rebuilding of a Docker container, rather than the reflashing of the hardware.

3.4 Software Simulator

For development sessions conducted without the physical board, `mqtt_publisher.py` was developed. This Python script, which runs within the `python-device` container, generates realistic temperature readings over MQTT. A sinusoidal baseline is utilized to simulate the natural thermal fluctuations of a motor, supplemented by random anomaly injections:

Listing 2: Temperature model in mqtt_publisher.py

```
r = random.random()
if r < 0.10:
    anomaly = random.uniform(10, 20)    # CRITICAL spike
elif r < 0.25:
    anomaly = random.uniform(4, 7)      # WARNING spike
else:
    anomaly = 0.0
base_temp = 65 + 3 * math.sin(t / 20) + noise + anomaly
```

The simulator also responds to fan commands: the temperature drops at 0.3 °C per cycle when the fan is on and the cooling offset decays at 0.1 °C per cycle when off. This closed-loop dynamic lets us verify the hysteresis logic and the predictive service without any real hardware.

4 Software Stack and Infrastructure

Written by: Janith Wanasinghe

4.1 MQTT Broker — Mosquitto

Eclipse Mosquitto 2.0 was configured as the MQTT broker. The configuration is maintained within `LDR-Hardware-simulation/mosquitto/mosquitto.conf` and is intentionally minimal: anonymous connections are permitted (suitable for a lab environment), and port 1883 is exposed to the host machine. Furthermore, persistent storage is enabled to ensure that retained messages—specifically the AUTO/MANUAL mode flag published by Node-RED—survive broker restarts. The exposure of port 1883 to the host is essential for the ESP32, which connects directly to the LAN IP address of the host machine over WiFi.

4.2 Node-RED HMI Dashboard

Node-RED serves as the operator interface. Core dashboard was built using the Dashboard v3 plugin. All HMI flows live in `node-red/flows.json` and are mounted into the Node-RED container via a bind mount on `./node-red:/data`.

The dashboard subscribes to two MQTT topics:

- `sensors/.../data` (raw temperature) — feeds the live temperature gauge and the historical trend chart.
- `alerts/.../status` (edge AI output) — feeds the alert status text, the alert log table, and the fan state indicator.

Two button nodes allow the operator to publish manual `ON` or `OFF` commands to the fan topic regardless of the current AUTO/MANUAL mode. A toggle switch publishes a retained message to the mode topic; because the message is retained, the edge AI container picks up the correct mode immediately on reconnect, even after a container restart.

The dashboard does not write to any external database at this stage — it is a self-contained

real-time HMI. Historical storage and the Grafana digital twin are handled by Chamodi as part of the predictive maintenance additions (Section 7).

4.3 Initial Docker Compose Structure

The initial `docker-compose.yml` for the `LDR-Hardware-simulation` directory was authored with the three services required to operate the core system: the Mosquitto broker, Node-RED, and the `python-edge` container. Each service is interconnected via Docker's internal bridge network, with service names utilized as hostnames to ensure the Python code remains independent of specific container IP addresses. The `python-edge` service is configured with `PYTHONUNBUFFERED=1`, allowing log output to be transmitted immediately to `docker compose logs` without buffering.

This three-service baseline was the foundation on which ML algorithm was added switching and the predictive service, InfluxDB, and Grafana were added.

5 Edge AI Module — Core Logic

Written by: Yohan Senanayake

5.1 Design Philosophy

The interface between the orchestration code and the classification logic was prioritized during the initial design of the edge AI module. To avoid tight coupling—which would necessitate multi-file modifications and increase the risk of regressions in fan control behavior—a two-function contract was established. This contract must be implemented by every algorithm, ensuring they remain easily interchangeable:

Listing 3: Module contract required by every classification algorithm

```python
def load_model() -> None:
    """Called once at container startup. Train or load here."""

def classify(temperature: float, window: deque) -> tuple[str,
   float]:
    """
    Returns (status, score).
      status : "NORMAL" | "WARNING" | "CRITICAL"
```

```
        score  : continuous numeric score for the MQTT alert
            payload
    """
```

With this contract in place, the entire swap is a single commented import line at the top of `edge_ai.py`. Nothing else in the orchestrator changes.

5.2 Rolling Window

The edge AI maintains a rolling window of the last 50 temperature readings using a `collections.deque(maxlen=50)`. The window fills up as readings arrive; once full, the oldest value drops off automatically. Every call to `classify()` receives both the current temperature and the full window, giving algorithms access to recent history.

A potential issue regarding small sample sizes was addressed: with fewer than 10 readings in the window, the standard deviation remains near-zero, which causes z-score computations to produce meaningless results. A `MIN_READINGS = 10` guard was implemented, ensuring no classification is attempted until the window contains at least 10 readings. During this startup period, incoming temperatures are logged while fan control actions are withheld.

5.3 Fan Control Hysteresis

The fan control logic is deliberately conservative. A single WARNING or CRITICAL reading turns the fan on immediately and resets a consecutive-normal counter to zero. The fan does not turn off until five consecutive NORMAL readings are observed:

Listing 4: Hysteresis logic in edge_ai.py

```
if status in ("WARNING", "CRITICAL"):
    current_fan_command = "ON"
    consecutive_normal = 0
else:
    consecutive_normal += 1
    if consecutive_normal >= FAN_OFF_CONSECUTIVE_NORMAL:  # 5
        current_fan_command = "OFF"
```

The reason for this design is relay chatter. Without hysteresis, a temperature oscillating around the warning threshold — say, 84–86 °C — would drive the fan on and off on alternate readings. That kind of rapid switching stresses the relay contacts and can also

destabilise the temperature itself by alternating between rapid cooling and re-heating. Five consecutive NORMAL readings gives the system confidence that the temperature has genuinely settled.

5.4 AUTO and MANUAL Mode

The operating mode is published by Node-RED as a retained MQTT message. Retained messages are delivered immediately to any new subscriber, so the edge AI always recovers the correct mode after a restart without operator intervention.

In AUTO mode (the default), the edge AI publishes a fan command after every classification. In MANUAL mode, it still classifies and logs every reading but does not publish fan commands — the operator drives the fan directly through the dashboard. This is useful during maintenance or when an operator wants to observe algorithm decisions without them taking effect.

5.5 Alert Payload

Every classification produces an alert published to both the local and cloud MQTT brokers:

Listing 5: Alert payload published on every reading (edge_ai.py)

```
{
    "status":       "WARNING",
    "temperature": 87.42,
    "z_score":      2.3145,     # score field; meaning depends on
       active algorithm
    "fan_command": "ON",
    "timestamp":    1714232100.0
}
```

The field is named `z_score` for backward compatibility with the Node-RED flow and InfluxDB schema, even when a non-Z-score algorithm is active. Changing the field name would require updating the Node-RED function nodes and all downstream queries, so we kept the name and documented what the value represents for each algorithm.

6 Reactive Classification Algorithms

Written by: Yohan Senanayake

6.1 Why Four Algorithms?

The initial objective for the classification module was the implementation of a single, optimal algorithm. However, it was determined that no universal solution exists, as the appropriate selection depends on system uptime, the prioritization of interpretability versus accuracy, and the availability of labeled historical data. Consequently, four interchangeable algorithms were developed behind a unified two-function contract. This architecture allows an operator to switch between detection methods by modifying a single import line and rebuilding the container.

6.2 Shared Feature Engineering

All three supervised algorithms use the same five-feature vector, centralised in `ml/base.py`. Having a single source of truth here matters: if the features used at training time differ from those used at inference time, the model will produce garbage outputs without any obvious error message.

Table 4: Five-feature vector computed from each reading

Feature	**Formula**	**Why it matters**
`temperature`	T	Absolute level
`z_score`	$(T - \mu_w) / \sigma_w$	Deviation relative to recent history
`delta`	$T - T_{\text{prev}}$	Rate of change
`rolling_mean`	μ_w over last 50 readings	Recent operating baseline
`rolling_std`	σ_w over last 50 readings	Recent variability

The `delta` feature was incorporated as it is considered to provide the most significant practical improvement over raw-temperature approaches. A motor at 83, C rising at 2, C per reading is identified as more hazardous than a motor at 83, C that has remained stable

for five minutes. While these two situations cannot be distinguished by a threshold-only system, they are differentiable by a model that accounts for `delta`.

6.3 Algorithm 0 — Z-Score Baseline

This is the original classification logic, extracted verbatim from the first version of `edge_ai.py` and wrapped in the module contract. It computes the z-score of the current reading relative to the rolling window:

$$z = \frac{T - \mu_{50}}{\sigma_{50}} \tag{2}$$

and applies hard absolute thresholds from `config.py`:

$$\text{status} = \begin{cases} \text{CRITICAL} & \text{if } T > 90\,^\circ\text{C} \\ \text{WARNING} & \text{if } T > 85\,^\circ\text{C} \\ \text{NORMAL} & \text{otherwise} \end{cases} \tag{3}$$

The z-score is computed and returned as the score field for logging, but does not influence the classification decision — that is entirely threshold-driven. The `load_model()` function for this module is a no-op stub that exists only to satisfy the module contract. This module serves as the regression baseline: if any other algorithm produces worse results than the Z-score on known test scenarios, something is wrong with that algorithm.

6.4 Algorithm 1 — Decision Tree

A Decision Tree recursively partitions the five-dimensional feature space by choosing binary splits that minimise class impurity (Gini index). The resulting tree is a set of explicit if-else rules that can be printed to the console and read by a human — something which was found genuinely useful when debugging edge cases in the classifier output.

Training strategy. Since no labelled field data exists, I generate synthetic training data from three Gaussian regimes representing normal operation, moderate overheating, and severe overheating:

Table 5: Synthetic training regimes for supervised algorithms

Class	**Mean**	σ	**Proportion**
NORMAL	77 °C	3.0	70%
WARNING	87 °C	2.0	20%
CRITICAL	93 °C	1.5	10%

Ten thousand samples are generated, with each sample labeled according to the same threshold rules as the Z-score baseline. The model is bootstrapped from the existing rule set, providing it the capacity to learn joint feature patterns—such as the observation that a moderate temperature combined with a high `delta` constitutes a stronger warning signal than either feature alone.

The classifier is configured with `max_depth=6` and `class_weight=''balanced''`. After some experiments, depth of 6 was chosen: shallower trees could not capture the interaction between `delta` and temperature; deeper trees started to memorise the synthetic distribution. The balanced class weight compensates for the 70/20/10 imbalance in the training data. Training takes about one second on first startup and the model is persisted to `models/decision_tree.joblib`; subsequent startups load from disk in milliseconds.

6.5 Algorithm 2 — Isolation Forest

Isolation Forest is the only unsupervised option in the set. It requires no labels whatsoever. Instead of learning from examples of anomalies, it learns what "normal" looks like and flags anything that deviates from that.

The algorithm builds 100 random trees. For each sample, it measures the average number of splits needed to isolate that point from the rest of the data. Anomalies are isolated quickly (short paths); normal points are buried deep among neighbours (long paths). The output of `decision_function()` is a continuous score: positive values indicate normal behaviour, negative values indicate anomalies.

Warm-up and self-calibration. There is no offline training step. The first 200 live readings are collected into a buffer and the model fits itself automatically once the buffer is full. The class boundaries are derived from the score distribution of those 200 readings: the 5th percentile becomes the CRITICAL threshold, the 15th becomes the WARNING

threshold. This means the thresholds adapt automatically to whatever the normal operating range of this particular piece of hardware turns out to be — a property the other three algorithms do not have.

Listing 6: Threshold calibration in isolation_forest.py

```
scores       = _model.decision_function(X)
_thresh_crit = float(np.percentile(scores, 5))
_thresh_warn = float(np.percentile(scores, 15))
```

The model also refits every 1 000 readings on a rolling 5 000-reading buffer. This periodic recalibration means it follows slow hardware drift over time without any manual intervention. During the ≈400-second warm-up period the module falls back to the Z-score hard thresholds, so the system is never unprotected.

6.6 Algorithm 3 — Random Forest

Random Forest is a direct upgrade to the Decision Tree. Rather than growing a single tree, it grows 100 trees in parallel, each trained on a bootstrap sample of the data and a random subset of features per split. The final prediction is the majority vote across all trees, and per-class probabilities are the average vote fractions.

The most useful practical difference from the Decision Tree is that I can tune sensitivity without retraining by adjusting two probability thresholds:

$$\text{status} = \begin{cases} \textsc{critical} & P(\text{CRITICAL}) \geq 0.50 \\ \textsc{warning} & P(\text{WARNING}) \geq 0.40 \\ \textsc{normal} & \text{otherwise} \end{cases} \tag{4}$$

The WARNING threshold is intentionally lower than 0.5 to catch borderline cases early. A model that only fires WARNING when it is more than 50% confident will sometimes miss genuine early-warning events. Lowering the threshold to 0.40 catches more edge cases at the cost of a small increase in false positives — a reasonable trade-off for safety-critical fan control.

6.7 Summary Comparison

Table 6: Side-by-side comparison of the four reactive algorithms

	Z-Score	**Decision Tree**	**Isolation Forest**	**Random Forest**
Training	None	Auto, ∼1 s	Auto, ∼400 s live	Auto, ∼3–5 s
Labels needed	No	No (synthetic)	No	No (synthetic)
Adapts to drift	No	No	Yes (periodic refit)	No
Interpretability	Highest	High	Low	Medium
Edge memory	<0.1 MB	<1 MB	∼5 MB	∼10 MB
Best for	Fallback	Explainable rules	Unknown baseline	Robust production

7 Predictive Maintenance Service

Written by: Chamodi Senarathne

7.1 Motivation and Scope

While the reactive edge AI addresses whether a motor is currently overheating, the objective was shifted to address a predictive concern: whether the warning threshold will be crossed within the next few minutes based on data reported over the preceding 60 seconds. By determining that approximately 8 minutes remain before a potential failure, proactive intervention is enabled. This allows for actions such as slowing a production line, dispatching a technician, or increasing monitoring frequency to be performed.

The scope of the project was expanded to include the storage and visualization layer that was previously absent from the `LDR-Hardware-simulation` stack. While the original `docker-compose.yml` consisted of only three services (MQTT, Node-RED, and `python-edge`) without persistent storage, several components were integrated: InfluxDB for time-series history, Grafana for the digital twin dashboard, the `python-predict` container for predictive services, and the `python-device` software simulator under a `simulation` profile. These additions were implemented without modifying the existing services, ensuring that the original Node-RED dashboard and edge AI container remained fully operational and unchanged throughout the process.

Design principle throughout was non-intrusiveness. The predictive service subscribes to the sensor topic in a read-only fashion and publishes only to its own new MQTT topic. It never touches the fan control topic. Each work could be developed and tested completely independently of the rest of the team.

7.2 New MQTT Topic and Output

The predictive service publishes to `predict/group10/motorTemp/risk` after every sensor reading, once the buffer has enough history. The payload contains four fields:

Listing 7: Predictive service output payload

```
{
    "risk_score":  0.73,        # 0.0 = no risk, 1.0 = imminent
       anomaly
    "eta_minutes": 4.2,         # estimated minutes to
       T_WARNING; null if not trending up
    "model":       "lstm",      # name of the active model
    "timestamp":   1714232100.0
}
```

7.3 Sliding Buffer

At the core of the service is a `SlidingBuffer(maxlen=30)` that accumulates the last 30 temperature readings. At one reading every 2 seconds, this represents 60 seconds of history. The service waits until the buffer contains at least 10 readings before producing any output, for the same reason Haritha uses a minimum window size in the edge AI module.

7.4 Phase 1 — Baseline Model (Linear Slope Extrapolation)

Prior to training any neural networks, a simple model was required to verify the entire pipeline—encompassing MQTT subscription, buffer management, inference, and result publication to the dashboard—without the need for training time or external model files. A baseline was established by fitting a linear regression to the last 15 readings, with the resulting trend extrapolated to the warning threshold:

Listing 8: Core prediction logic in predictive/baseline_model.py

```
def predict(buf: SlidingBuffer) -> dict:
    if len(buf) < 10:
```

```
        return {"risk_score": 0.0, "eta_minutes": None,
           "model": "baseline"}
    readings = np.array(buf.latest(15))
    slope    = extract_slope(readings)   # deg C per reading
    current  = readings[-1]
    if slope > 0 and current < T_WARNING:
        readings_needed = (T_WARNING - current) / slope
        eta_minutes     = (readings_needed * 2) / 60.0
    else:
        eta_minutes = None
    risk_score = float(np.clip(slope / MAX_SLOPE, 0.0, 1.0))
    return {"risk_score": risk_score, "eta_minutes":
       eta_minutes, "model": "baseline"}
```

This model is simple enough to be reasoned about entirely on paper, which proved invaluable when the dashboard integration was being debugged. If the risk gauge did not respond as expected, the raw slope value could be examined to immediately determine the cause.

7.5 Phase 2 — LSTM Model

The linear baseline has a fundamental limitation: it only looks at the slope of the last 30 seconds. A motor that has been slowly warming for ten minutes and is now at 80 °C carries a fundamentally different risk profile from one that spiked to 80 °C two readings ago. An LSTM can learn these temporal patterns because it maintains a hidden state across the entire input sequence.

Architecture

The network takes a sequence of 30 time steps, each described by three features (temperature, delta, z-score). A 64-unit LSTM layer extracts temporal patterns, a Dropout(0.2) layer reduces overfitting, and two dense layers produce the final risk probability:

Listing 9: LSTM model definition in predictive/lstm_model.py

```
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(64, input_shape=(30, 3)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(32, activation='relu'),
```

```
    tf.keras.layers.Dense(1,  activation='sigmoid'),
])
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy',
                  tf.keras.metrics.AUC(name='auc')])
```

The output is a single probability value: the model's estimate that the motor will cross `T_WARNING` within the next 10 readings (20 seconds). A value above 0.5 is treated as a risk event.

Training Locally, Not in Docker

A deliberate decision was made to train the LSTM locally using the existing `uv` environment rather than within a Docker container. This approach was justified by the fact that the full TensorFlow library adds approximately 900,MB to the Docker image, whereas training is only performed once. By avoiding the installation of TensorFlow within Docker for a single offline training task, the duration of subsequent `docker compose up -build` operations was minimized for all team members, without impacting performance at inference time.

The trained model is saved as a `.keras` file and mounted into the container as a volume:

Listing 10: Model volume mount in docker-compose.yml

```
python-predict:
  build: ./python
  environment:
    - PREDICTIVE_MODEL=baseline   # change to "lstm" after
       training
  volumes:
    - ./models:/app/models
```

The container uses `tflite-runtime` (≈5 MB) for inference instead of full TensorFlow, keeping the image lean and startup time short.

Synthetic Training Data

In the absence of a labeled historical dataset, training sequences are generated programmatically using the simulation logic from `mqtt_publisher.py`. These sequences are produced by a generation script that runs the simulator in-process, bypassing the need for an MQTT

connection. A total of 50,000 labeled sequences are generated, with a sequence being labeled positive (risk = 1) if the temperature crosses `T_WARNING` within the subsequent 10 readings; otherwise, it is labeled negative.

Table 7: Class distribution in the LSTM training dataset

Class	**Proportion**	**Generation method**
NORMAL (no anomaly ahead)	65%	Clean sinusoidal window
WARNING-ahead	20%	WARNING spike within next 10 readings
CRITICAL-ahead	15%	CRITICAL spike within next 10 readings

Target evaluation metrics on a held-out 20% test split: AUC-ROC ≥ 0.95, accuracy ≥ 0.90, false positive rate $\leq 10\%$.

7.6 Swappable Model Registry

Both models share the same interface: a `predict(buf)` function that takes a `SlidingBuffer` and returns a result dictionary. The active model is selected by an environment variable at container startup, with no code change needed to switch:

Listing 11: Model registry in predictive/model_registry.py

```
ACTIVE_MODEL = os.getenv("PREDICTIVE_MODEL", "baseline")

def get_predictor():
    if ACTIVE_MODEL == "lstm":
        from .lstm_model     import predict
    else:
        from .baseline_model import predict
    return predict
```

7.7 InfluxDB, Grafana, and Node-RED Integration

The original three-service LDR stack lacked both persistent storage and a historical dashboard; therefore, these components were integrated as part of this development. InfluxDB 2.7 is initialized upon the first startup using environment variables within the

Docker Compose service definition. A fixed admin token is utilized, allowing other services to reference the database without requiring a manual setup step:

Listing 12: InfluxDB first-run init in docker-compose.yml

```
environment:
  - DOCKER_INFLUXDB_INIT_MODE=setup
  - DOCKER_INFLUXDB_INIT_ORG=group10
  - DOCKER_INFLUXDB_INIT_BUCKET=motor_thermal
  - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=motor-token-group10
```

A decision was made to hardcode the token rather than utilize a `.env` file, as the existing project structure did not employ one. Introducing that dependency was avoided to ensure the established workflow of the other team members remained unaffected. The token value is treated as non-secret within the context of a lab environment.

Two measurements are written to the `motor_thermal` bucket:

- `motor_alert` — written by a new Node-RED InfluxDB write node I added to the predictive flow, recording `temperature`, `score`, `fan_command`, and `status` from each edge AI alert.
- `motor_prediction` — written by the same flow from each predictive service message, recording `risk_score` and `eta_minutes` tagged with the active model name.

The predictive flows are kept in a separate file (`flows-predict.json`) so they do not merge conflict with Janith's `flows.json`. Node-RED automatically loads all `flows*.json` files from its data directory at startup.

A Grafana dashboard was provisioned to overlay the temperature trend with the LSTM risk score. The most functional component is the time-series overlay, where temperature is mapped to the left axis and the risk score (scaled to 0–100) is mapped to the right axis. Color bands are utilized to mark the WARNING and CRITICAL zones, ensuring that the early warning lead time is made visible at a glance.

7.8 Key Result — Early Warning Lead Time

The most compelling way to evaluate the predictive service is the lead time: the gap in seconds between when the LSTM risk score first crosses 0.5 and when the reactive edge AI fires its first WARNING. With the LSTM active and the forced-anomaly scenario:

1. The temperature begins climbing from the sinusoidal baseline.
2. The LSTM risk score crosses 0.5 approximately 30–60 seconds before the temperature reaches 85 °C.
3. The edge AI fires its first WARNING when the temperature crosses 85 °C.

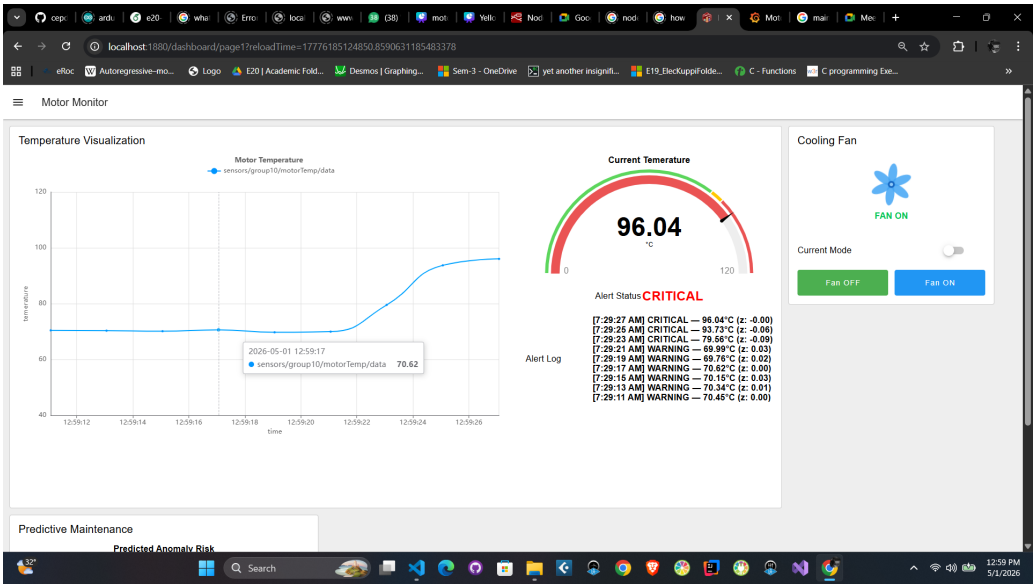

Figure 3: Demonstration of early warning lead time. The predictive alert (green) fires before the reactive WARNING (red), giving the operator advance notice.

Achieving a lead time of 30 seconds or more validates the core claim: temporal pattern analysis can anticipate a failure that threshold-based monitoring cannot.

8 Challenges and Solutions

Every team member encountered unexpected obstacles during development. This section documents the main problems identified and the solutions implemented.

8.1 ESP32 ADC2 Conflict with WiFi

Problem. The LDR was initially wired to GPIO 13, which produced erratic ADC readings that would occasionally spike or drop without any visible cause. **Root cause.** GPIO 13 belongs to the ESP32's ADC2 bank, which is shared with the WiFi radio. When WiFi is active, ADC2 readings become unreliable. **Solution.** Relocating the LDR to GPIO 34 (ADC1 bank) resolved the issue immediately. ADC1 operates independently of the WiFi peripheral.

8.2 Z-Score Instability on Cold Start

Problem. The edge AI occasionally fired spurious WARNING alerts within the first few seconds of receiving data, before the temperature had reached an anomalous level. **Root cause.** With only 2–3 readings in the window, the standard deviation is near-zero. Any reading differing even slightly from the window mean produces a very large z-score. **Solution.** A `MIN_READINGS = 10` guard was added in `z_score.py` and the feature extraction function in `base.py`. Classification is skipped entirely until at least 10 readings are present in the window.

8.3 Isolation Forest Silent Warm-Up

Problem. When the Isolation Forest was active, the system silently used the z-score fallback for the first 6–7 minutes without any indication in the logs. **Solution.** A per-reading progress log line was added during warm-up (e.g., `[IF ] Warm-up 47/200 -- using fallback`), along with a corresponding Node-RED status widget that displays "ML model warming up" until the model is fitted.

8.4 Training Data Generation and Class Imbalance

Problem. Purely random temperature simulation generated anomalies less than 1% of the time. The LSTM model achieved high accuracy simply by predicting "SAFE" continuously, resulting in a 0**Solution.** Sticky Regime-Based Sampling was implemented. The simulator draws from three distinct Gaussian distributions (Normal, Warning, Critical) and persists in each state for 30–100 readings, guaranteeing a balanced dataset (~35

8.5 Asynchronous Data vs. Sequential Input

Problem. MQTT data arrives asynchronously (one reading at a time), but the LSTM requires a fixed 30-reading sequential tensor to perform inference. **Solution.** A thread-safe SlidingBuffer adapter was developed. It queues incoming MQTT payloads, handles the required warm-up period, and safely provides the exact sequential window the model requires without data-race conditions.

8.6 InfluxDB Init Token Bootstrap

Problem. InfluxDB 2.x requires an API token for all writes. On first startup, no token exists yet, and the Node-RED write nodes would fail until the token was created manually

through the web UI. **Solution.** The `DOCKER_INFLUXDB_INIT_ADMIN_TOKEN` environment variable in the Docker Compose configuration was used to create the admin token at first run with a known value. Both Node-RED and the predictive service reference this fixed token directly, eliminating the need for any manual step.

8.7 Node-RED Flow Merge Conflicts

Problem. Adding predictive nodes to the existing `flows.json` caused Git merge conflicts with concurrent HMI development work. **Solution.** All predictive-related flows were moved into a separate `flows-predict.json`. Node-RED auto-merges all `flows*.json` files at startup, keeping both files independent in the repository such that changes to one do not affect the other.

8.8 tflite-runtime Architecture Compatibility

Problem. The `tflite-runtime` wheel on PyPI is not available for all Python/architecture combinations, causing the container build to fail on non-standard hosts. **Solution.** A graceful fallback was added in `lstm_model.py`: if the `tflite-runtime` import fails, the module attempts to fall back to full TensorFlow, or raises a clear error with installation guidance. The primary deployment target (x86,x64 Linux) functions correctly with `tflite-runtime` in all tested configurations.

9 Member Contributions

Written by: Haritha Bandara

Table 8: Work allocation across team members

Member	**Primary contributions**
Haritha Bandara	System architecture design; `edge_ai.py` (MQTT orchestration, dual-publish pattern, fan hysteresis logic, AUTO/MANUAL mode switching); `config.py`; Docker Compose coordination; cloud SCADA integration; overall project structure; report sections 1, 2, 5, 9, and 10.
Yohan Senanayake	`ml/` package: `base.py` (shared feature engineering, synthetic data generation), `z_score.py` (baseline extraction and module wrapping), `decision_tree.py`, `isolation_forest.py`, `random_forest.py`; warm-up logging and dashboard indicator for Isolation Forest; developer documentation (`ml/README.md`); report section 6.
Chamodi Senarathne	`predictive/` package (`buffer.py`, `features.py`, `baseline_model.py`, `lstm_model.py`, `model_registry.py`); `predictive_service.py`; offline LSTM training pipeline (`generate_training_data.py`, `train_lstm.py`); Node-RED predictive flow (`flows-predict.json`) including InfluxDB write nodes; InfluxDB service and data model; Grafana digital twin dashboard; Docker Compose additions; report section 7.
Janith Wanasinghe	Unified system architecture (`docker-compose.yml`): orchestration of the six-service stack; InfluxDB historian integration: container automated setup and Node-RED data routing pipeline; Grafana Digital Twin dashboard (`motor-health-dashboard.json`): custom Flux queries; automated Grafana datasource and dashboard provisioning; system integration and debugging; comprehensive system documentation; report sections 3 and 4.

10 Conclusion and Future Work

Written by: Haritha Bandara

10.1 What We Built

The objective of this project was to develop a motor thermal monitoring Digital Twin that goes beyond a simple alarm system, and the outcome demonstrates that this goal was achieved. Beginning from a three-service stack in the `LDR-Hardware-simulation` folder — MQTT, Node-RED, and a single Python edge AI container — the system was expanded into a full seven-service architecture incorporating real hardware sensing, four interchangeable ML classifiers, a predictive maintenance service backed by an LSTM, a time-series historian, and a Grafana digital twin dashboard. The gap in seconds observed between the predictive alert and the reactive WARNING serves as the clearest evidence of the value added by the temporal modelling layer over conventional threshold-based monitoring.

The two-layer AI architecture represents one of the most significant design decisions made throughout the project. Reactive and predictive intelligence are often considered alternatives; however, in this implementation they were found to complement each other effectively: the reactive module functions as an immediate safety net, while the predictive module provides operators with sufficient time to respond. The successful operation of both layers simultaneously from the same sensor stream — deployed in separate containers with clean interfaces that required minimal cross-team coordination — validates the suitability of this architecture for real industrial deployments.

10.2 Future Work

On-device inference. The ultimate edge deployment would run classification directly on the ESP32 using TensorFlow Lite for Microcontrollers, eliminating the Python containers for the reactive path. The five-feature vector is small enough to fit within the ESP32's 320 KB RAM.

Online learning. When a human operator confirms that a WARNING was genuine, that event could be added to the training set and the supervised models refined in the background, progressively improving accuracy on the actual hardware.

Multi-motor aggregation. The architecture scales naturally: each motor gets its own MQTT topic prefix and Docker Compose instance, and a single Grafana dashboard can query multiple InfluxDB buckets to show all motors side by side.

Security hardening. The current Mosquitto configuration accepts anonymous connec-

tions. Adding TLS and client certificates to the broker, with proper secret management for the InfluxDB token, would bring the system to a production-deployable state.

RUL estimation. With enough historical data in InfluxDB, it becomes possible to estimate Remaining Useful Life from the trend in alert frequency and the rate at which the motor's thermal baseline drifts upward over months — a richer signal than any individual reading can provide.

References

[1] EPRI, "Electric Motor Predictive Maintenance: Draft Guidelines." Available: https://restservice.epri.com/publicdownload/TR-108773-V1/0/Product

[2] Espressif Systems, "Analog to Digital Converter - ESP32." Available: https://docs.espressif.com/projects/esp-idf/en/v4.2/esp32/api-reference/peripherals/adc.html

[3] Eclipse Mosquitto, "Documentation." Available: https://mosquitto.org/documentation/

[4] Node-RED, "Documentation." Available: https://nodered.org/docs/

[5] Node-RED, "Node-RED Dashboard." Available: https://flows.nodered.org/node/node-red-dashboard

[6] Grafana Labs, "InfluxDB data source." Available: https://grafana.com/docs/grafana/latest/datasources/influxdb/

[7] Grafana Labs, "Get started with Grafana and InfluxDB." Available: https://grafana.com/docs/grafana/latest/fundamentals/getting-started/first-dashboards/get-started-grafana-influxdb/

[8] "Predictive Maintenance of Motors using Machine Learning." Available: https://www.ijnrd.org/papers/IJNRD2404282.pdf

[9] "Health Monitoring and Predictive Maintenance of DC Motor." Available: https://www.isca.me/IJES/Archive/v14/i1/2.ISCA-RJEngS-2024-014.pdf

[10] "Predictive Maintenance for Electric Motors in IoT-Enabled Mechatronic Systems." Available: https://www.politesi.polimi.it/retrieve/58556b14-9543-4e9b-8ea5-5dd21b63fc11/2024_12_Ampeli.pdf